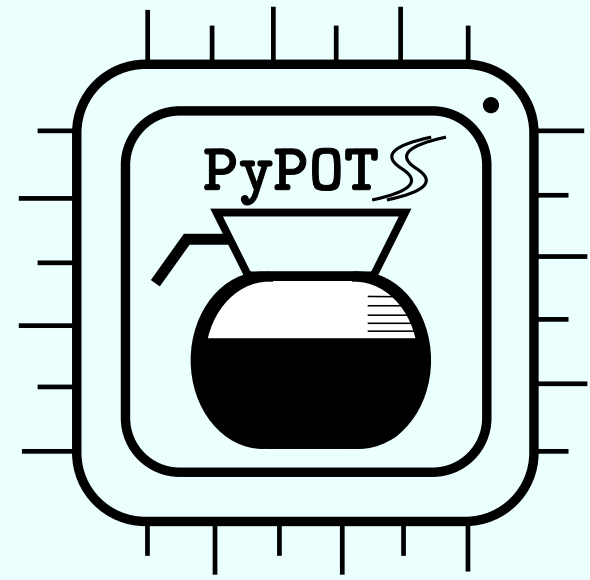




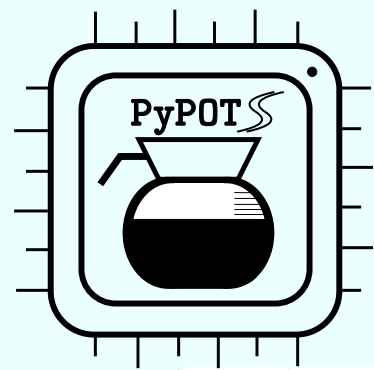
Learning from POTS: Towards Reality-Centric AI for TS

Keynote for IJCAI'24 AI4TS (AI for Time Series Analysis) Workshop

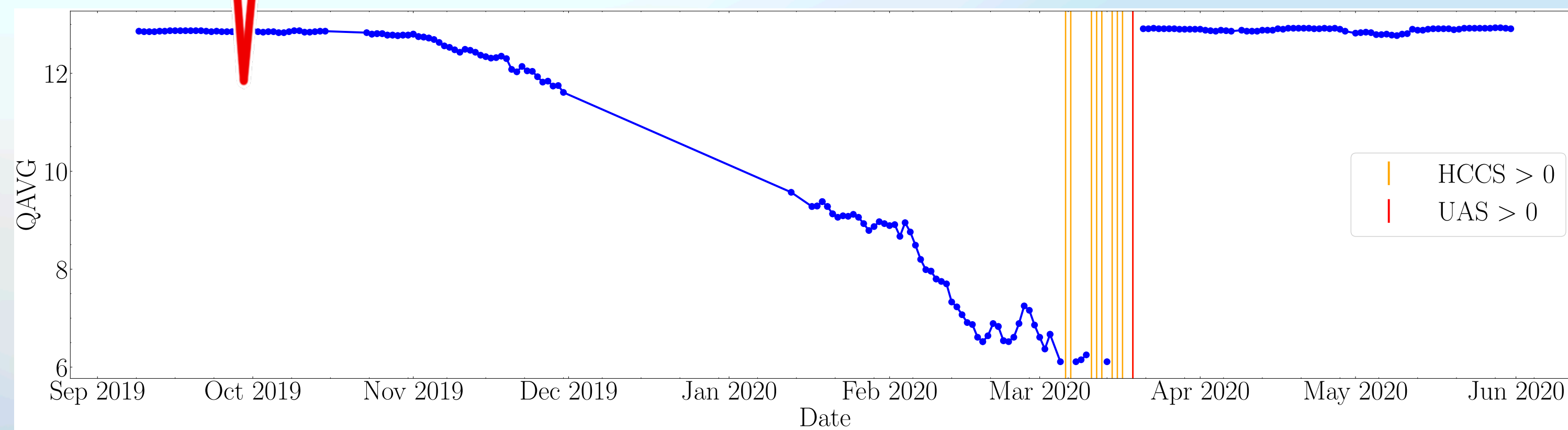
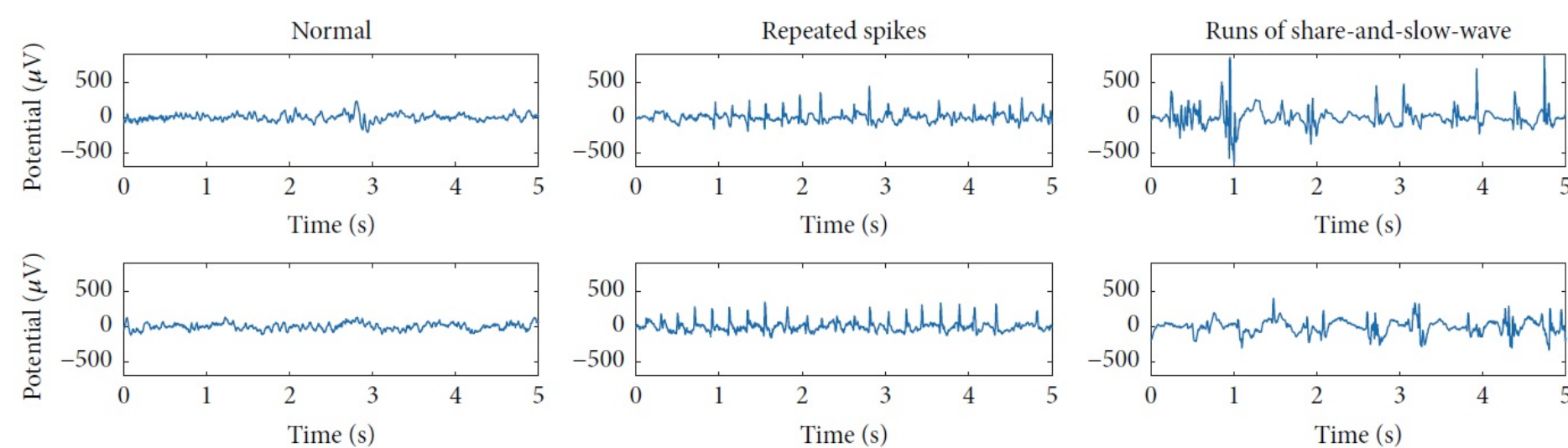
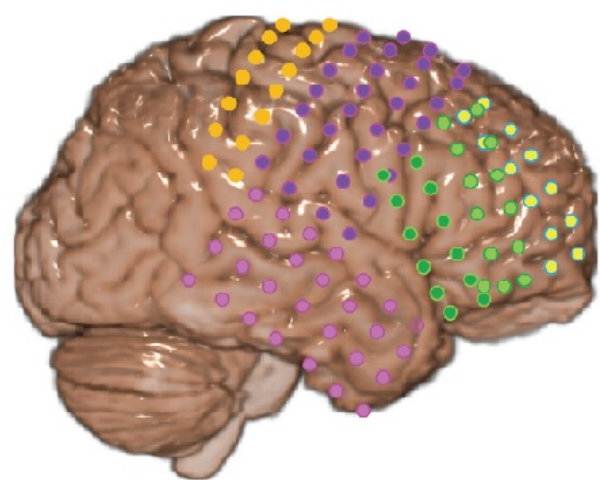
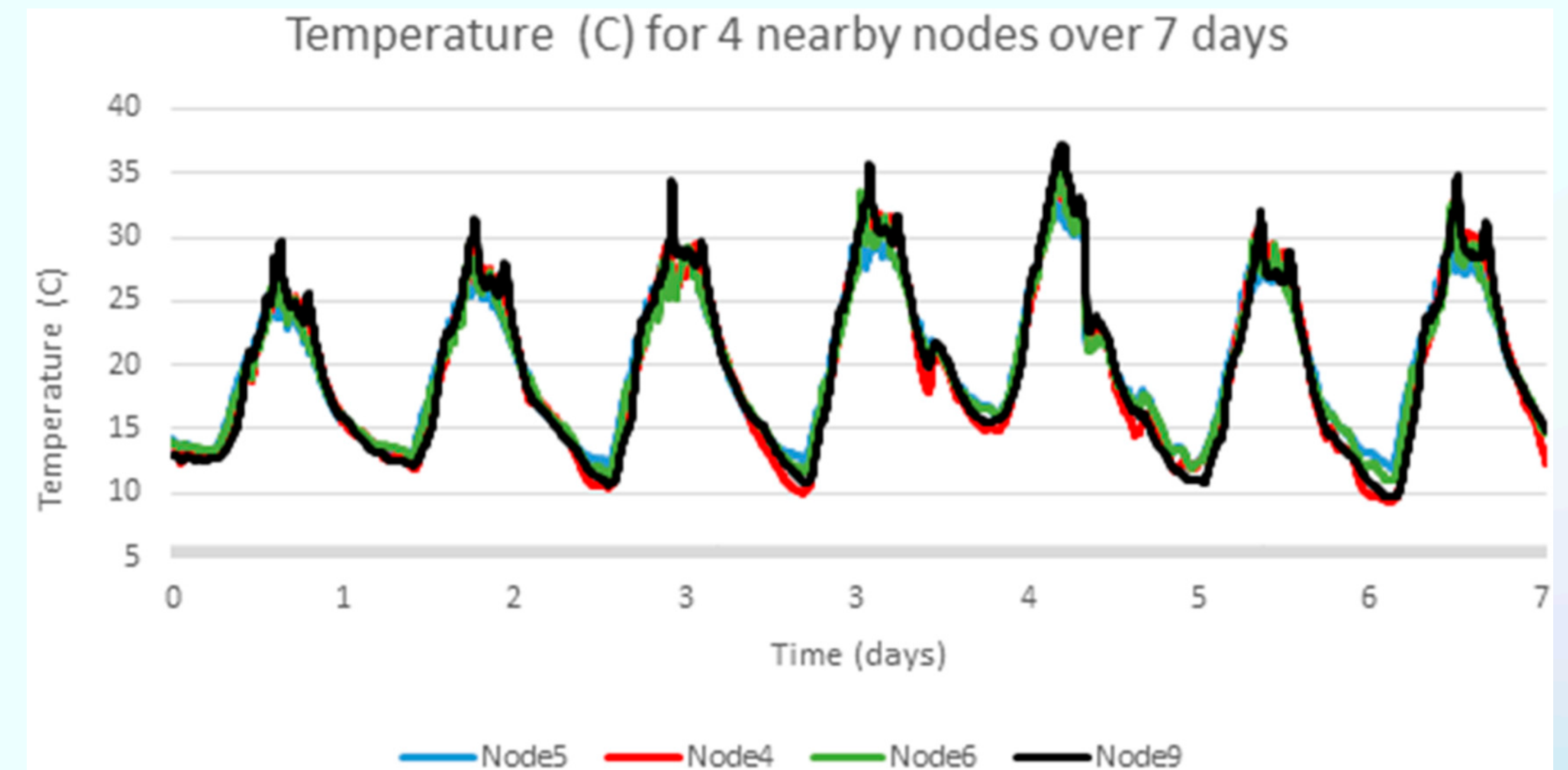
Wenjie Du <wdu@pypots.com>

PyPOTS Research

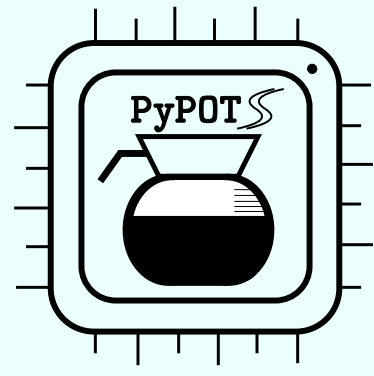
2024 Aug 05



Time series is everywhere



<https://pkai.pknu.ac.kr/index.php/2020/01/21/eeg-signal-processing/>



POTS (Partially observed time series)

1. Originally missing values

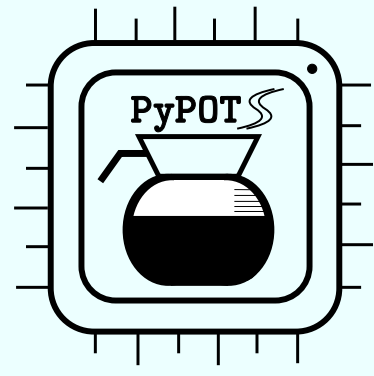
- failed sensor;
- Interrupted network;

2. Created missing values

- anomaly detection then exclusion;

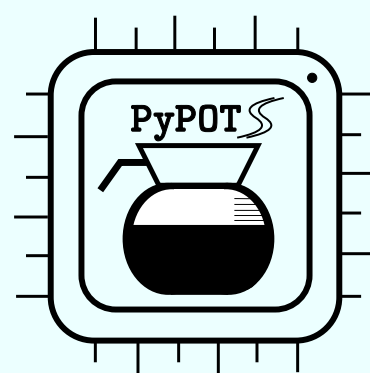
3. Irregular sampling

- alignment of samples with different time granularity;



Hard to learn from POTS

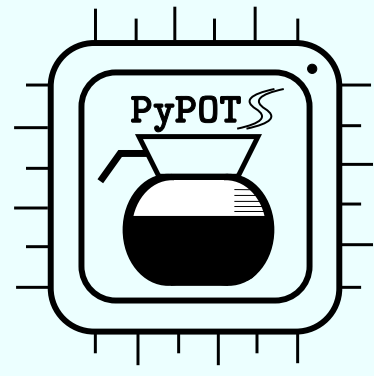
1. Most time series algorithms not applicable because typically assume complete data input;
2. Missing part could be meaningful but hard to represent;
3. Imputation can help but is computationally intensive and needs dedicated design^[1];



Reality-Centric AI [1]

	Petri-dish AI	Reality-centric AI
Domains	Successfully and impressively plays games and solves straightforward-to-formalise scientific-discovery problems	Focusses on problems in complex, human-centric domains such as medicine, education, finance, and business
Machine learning methods	Dominated by reinforcement learning	Pragmatic about methods
How the model maps to the world	Chooses domains where the model mapping of the world is straightforward	Makes important-for-the-model's-outcome decisions about how to <i>design</i> a model's abstractions of, and assumptions about, the (messy and complex) world that are sufficiently well defined for machine learning to operate effectively
Data	Needs lots of accurate and inexpensive data	Expects deficient data, which if available at all, is often expensive to collect
Environment	Assumes fixed environments	Accommodates changing environments
Access to information about the environment	Assumes perfect information about the environment	Expects imperfect and incomplete information about the environment

[1] Mihaela van der Schaar, and Andrew Rashbass. 2023. "The case for Reality-centric AI". <https://www.vanderschaar-lab.com/the-case-for-reality-centric-ai/>



The definition of POTS [1]

Given a time series (already aligned if irregularly sampled)

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D) \in \mathbb{R}^{L \times D}$$

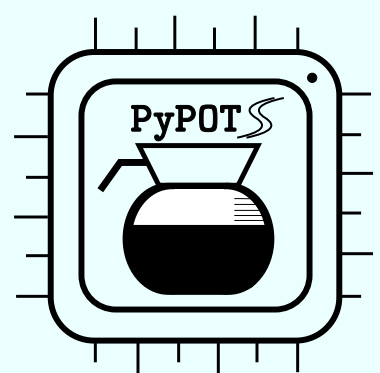
where L is the data length;

D is the data dimension, i.e the number of features;

Any data point $\mathbf{X}_{t,d}$ in the time series could be missing, hence a mask matrix is defined to represent the observation status of data points

$$\mathbf{M} \in \mathbb{R}^{L \times D}$$

Where $\mathbf{M}_{t,d} = 1$ if $\mathbf{X}_{t,d}$ is observed, otherwise $\mathbf{M}_{t,d} = 0$

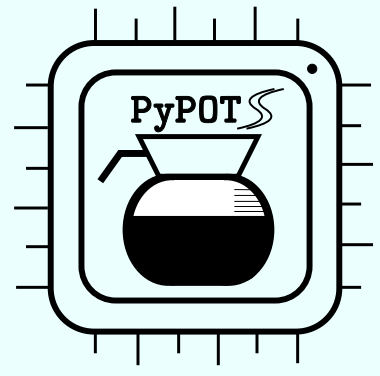


Imputation

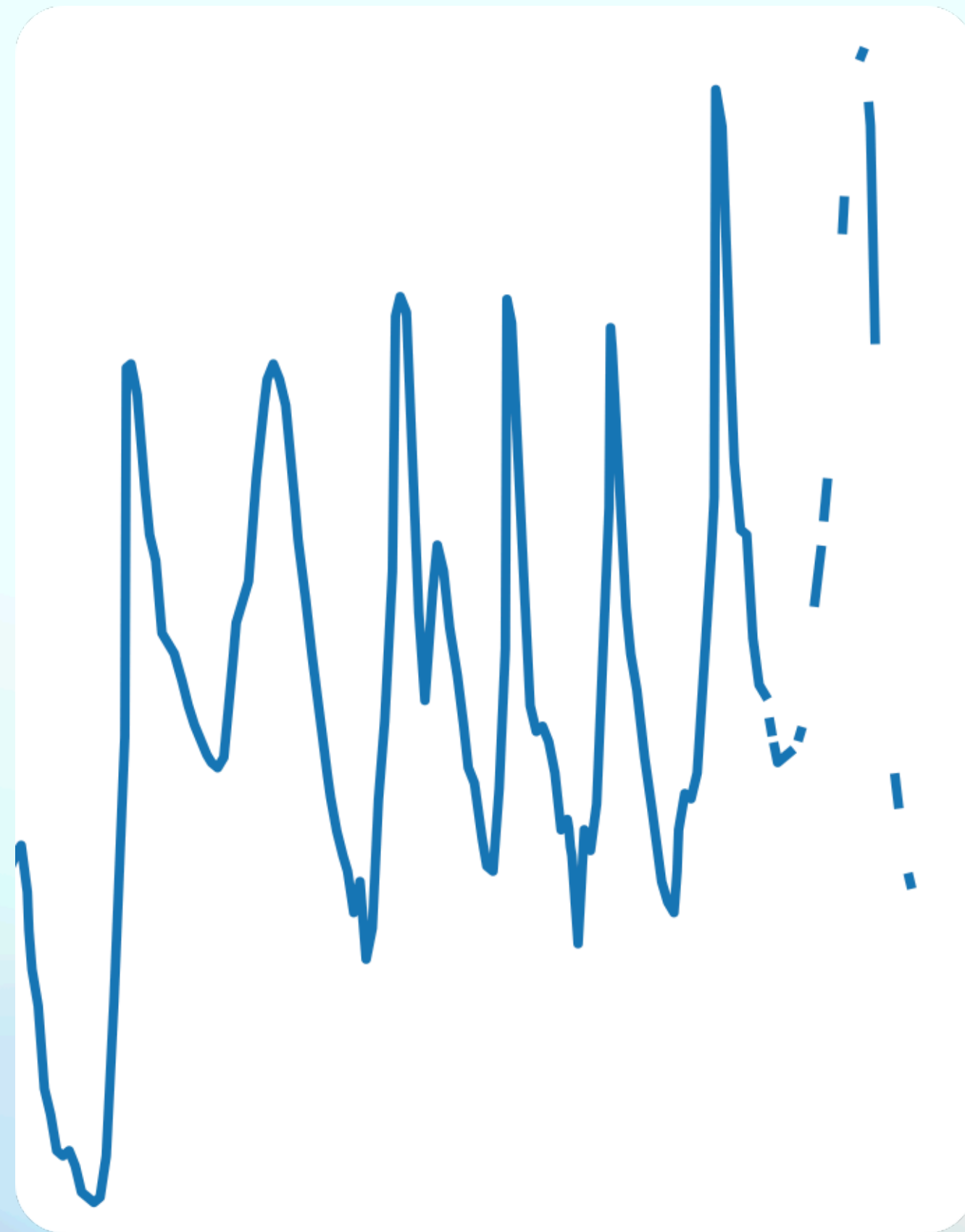
	col1	col2	col3	col4	col5
0	2	5.0	3.0	6	NaN
1	9	NaN	9.0	0	7.0
2	19	17.0	NaN	9	NaN

`df.fillna(0)`

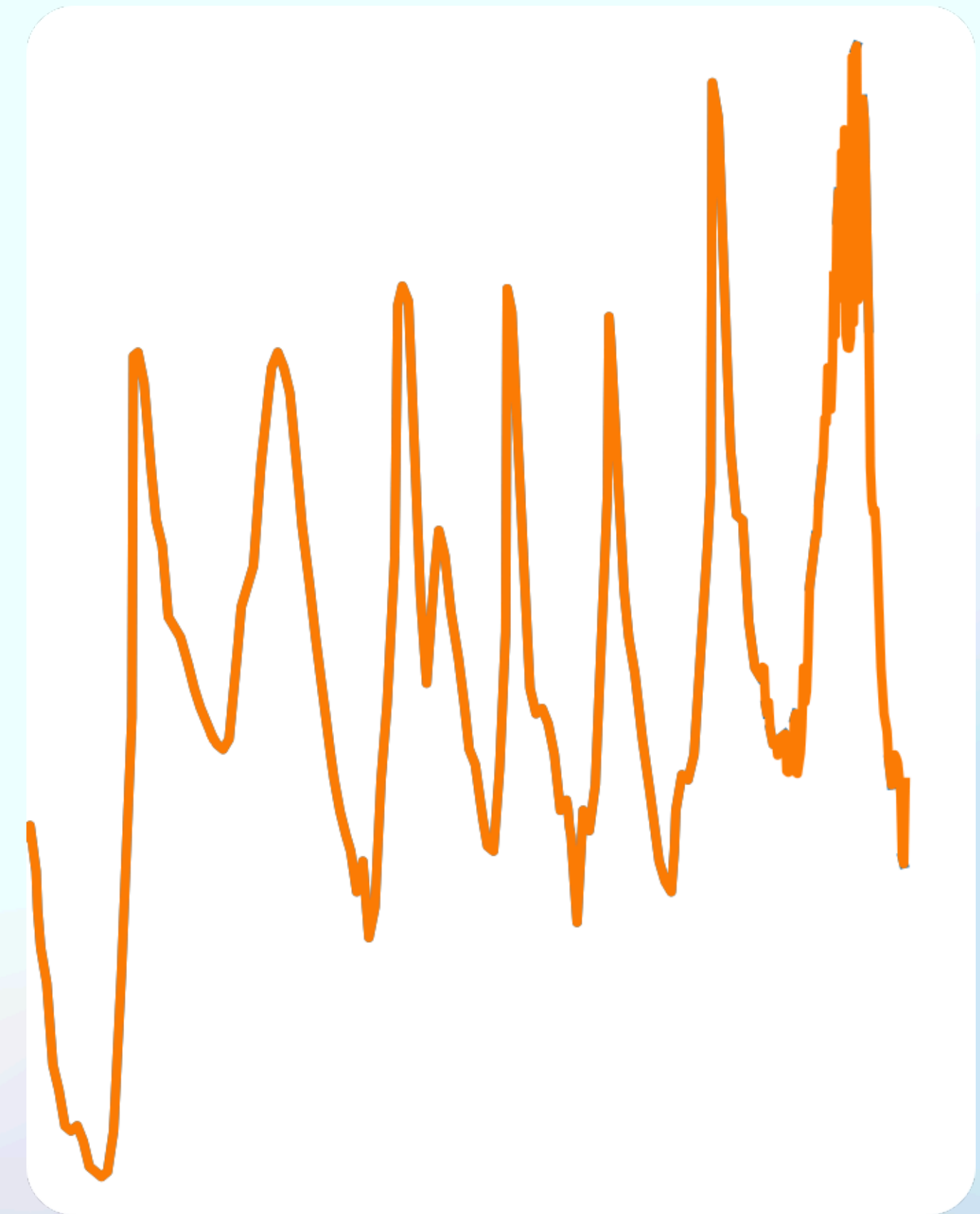
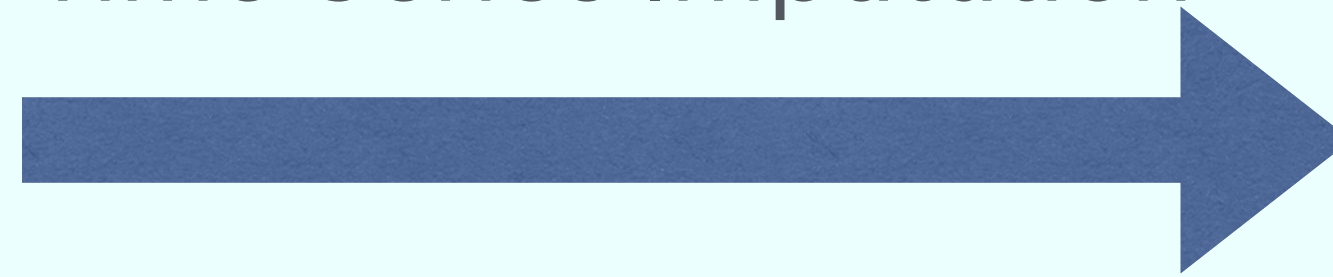
	col1	col2	col3	col4	col5
0	2	5.0	3.0	6	0.0
1	9	0.0	9.0	0	7.0
2	19	17.0	0.0	9	0.0

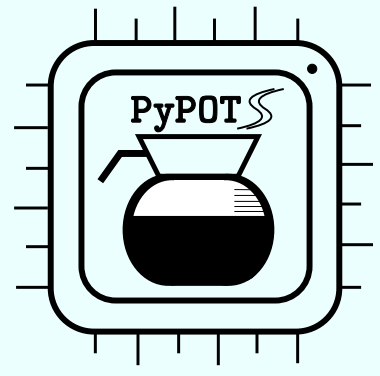


Imputation



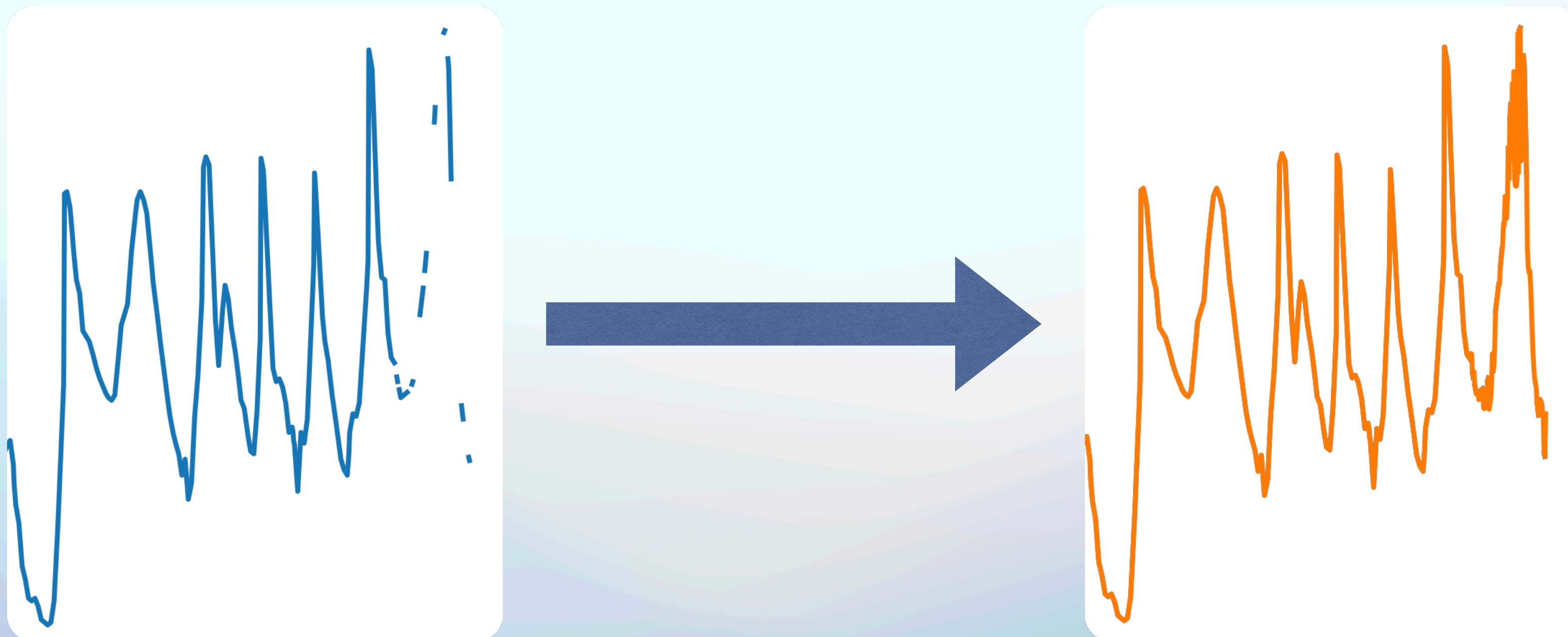
Time Series Imputation

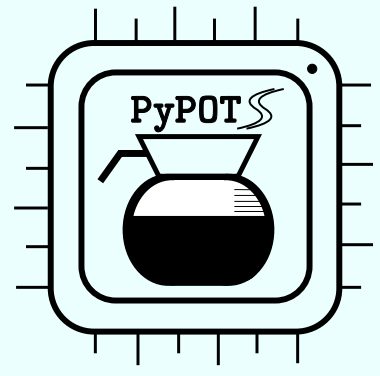




Imputation, a vital step of preprocessing

Fill the missing data with values meaningful in the context can help fill the gap between complete time series analysis models and incomplete time series

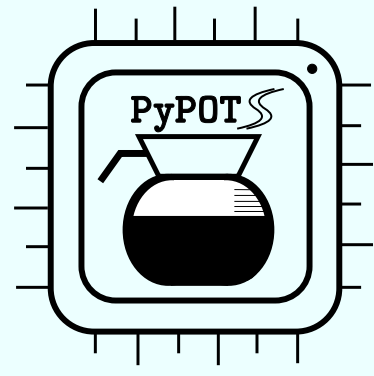




The definition of imputation

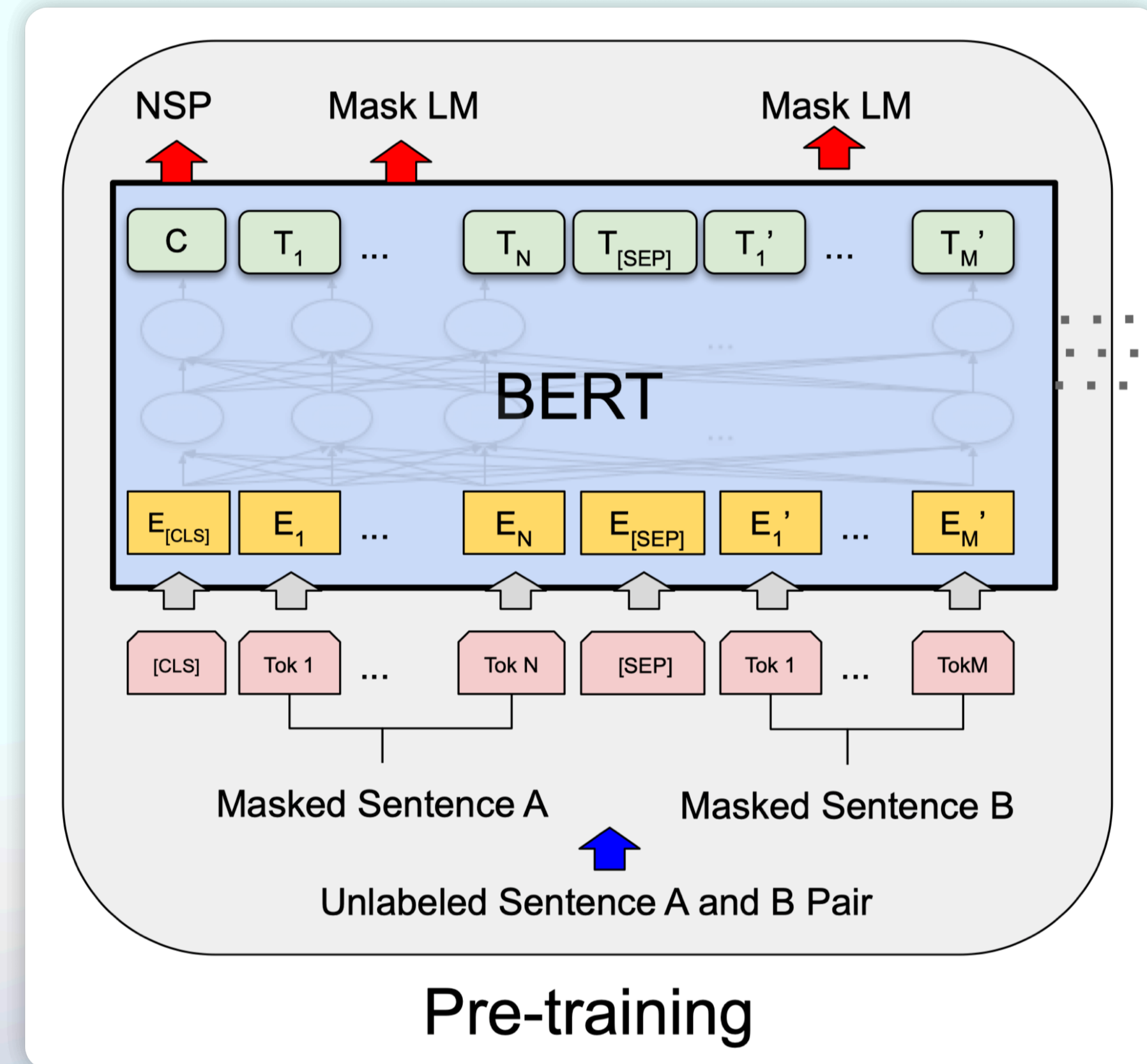
Given the partially observed time series X ,
the objective of time series imputation is to obtain
the imputed time series X' with the reconstruction $\tilde{X}$

$$\mathbf{X}' = \mathbf{M} \odot \mathbf{X} + (1 - \mathbf{M}) \odot \tilde{\mathbf{X}},$$

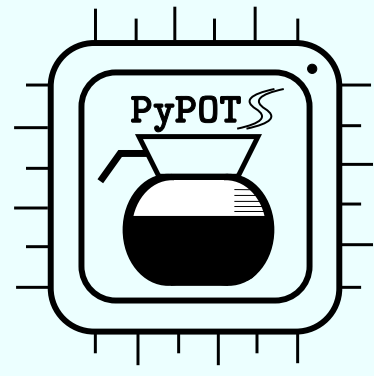


Imputation, another form of the Cloze task^[1]

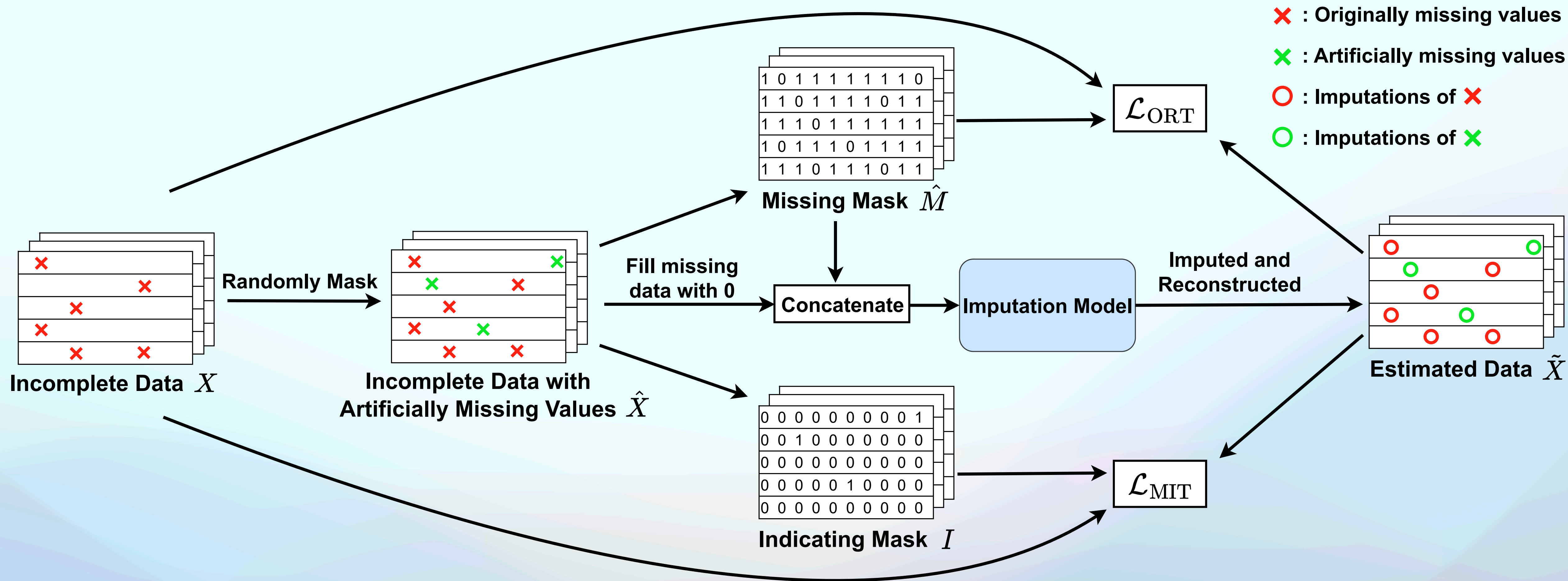
When it comes to time series, the Cloze task, also known as the Masked LM task presented in BERT, turns out to be an imputation task.

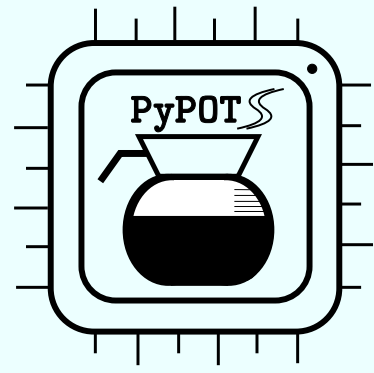


[1] Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. Journalism Bulletin, 30(4):415–433.

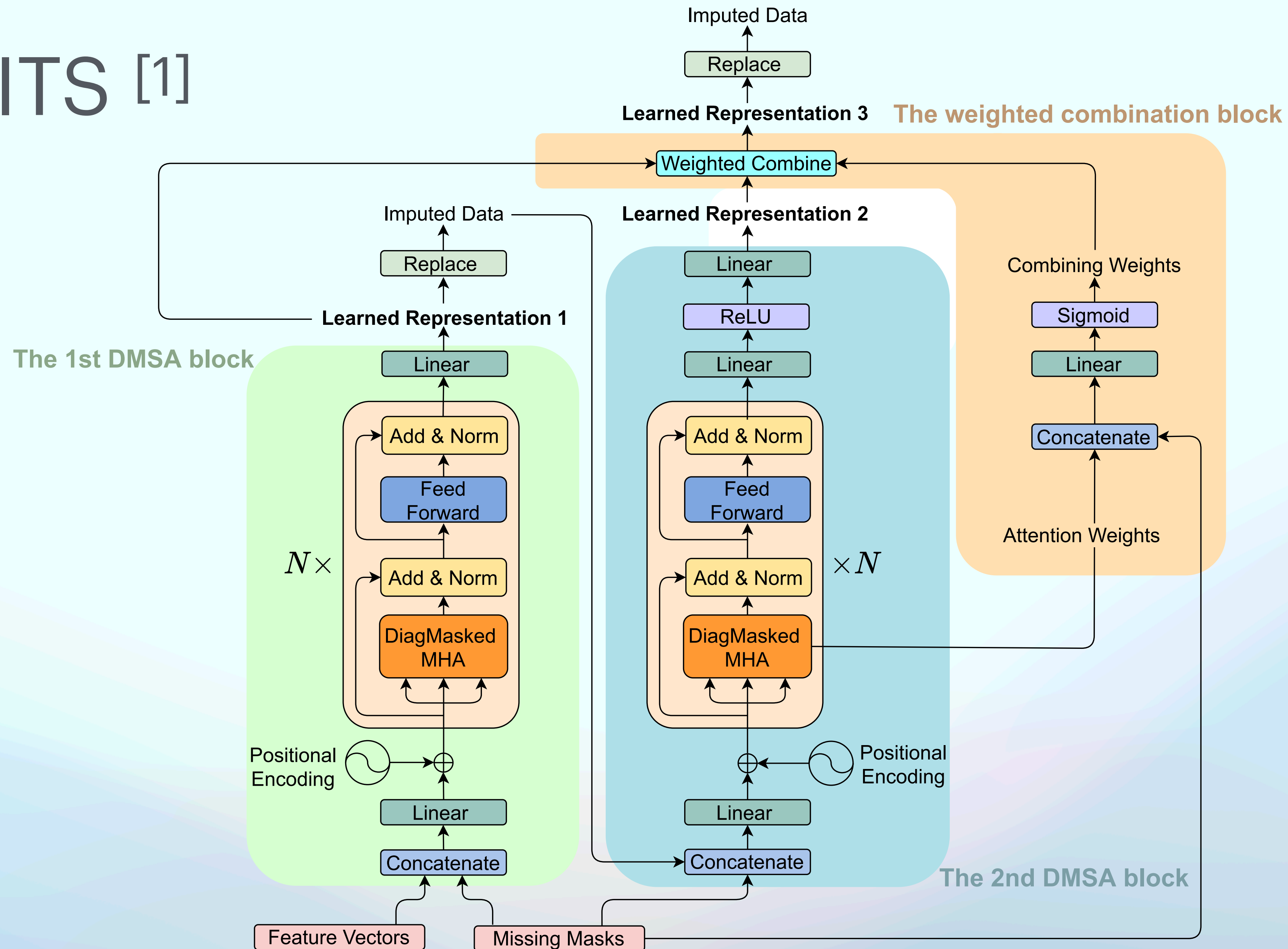


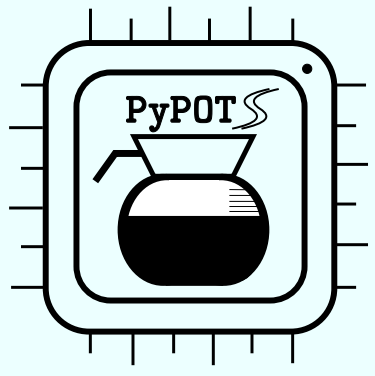
Masked imputation task from SAITS [1]





SAITS [1]



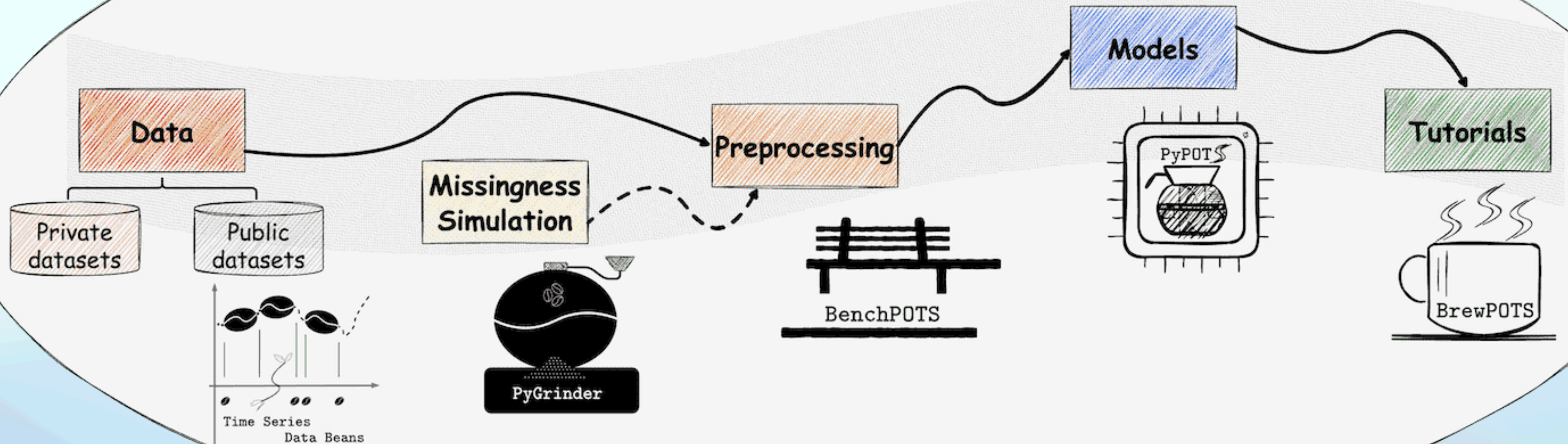


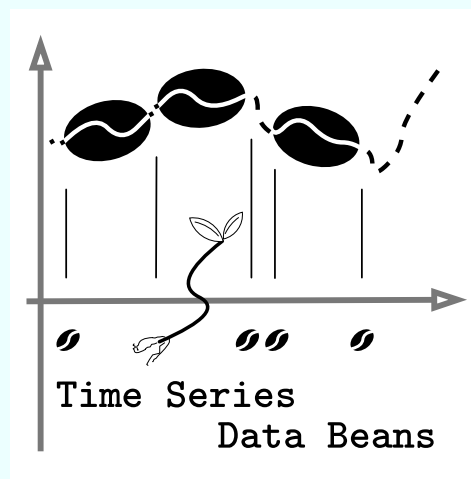
PyPOTS Ecosystem <https://pypots.com>

Why? Data. Things start from data.

What? Models. What models/algorithms can help.

How? Tutorials. Guide people to engage with data and models.





TSDB (Time Series Data Beans)

A Python library helps load **172** public time series datasets in a single line

A quick example:

```
>>> import tsdb  
>>> data_dict=tsdb.load('physionet_2012')
```

2024-08-01 13:36:02 [INFO]: You're using dataset physionet_2012, please cite it properly in your work. You can find its reference information at the below link:

https://github.com/WenjieDu/TSDB/tree/main/dataset_profiles/physionet_2012

2024-08-01 13:36:02 [INFO]: Dataset physionet_2012 has already been downloaded. Processing directly...

2024-08-01 13:36:02 [INFO]: Dataset physionet_2012 has already been cached. Loading from cache directly...

2024-08-01 13:36:02 [INFO]: Loaded successfully!

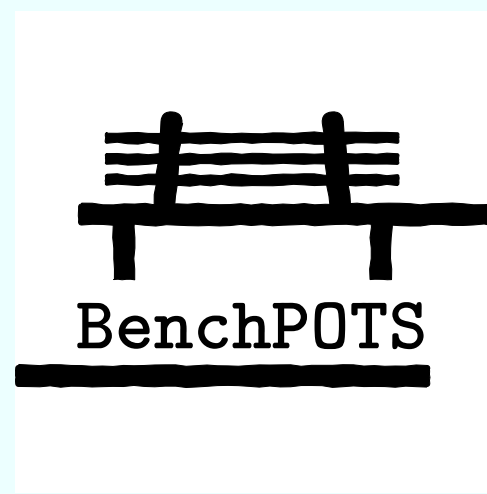


PyGrinder

A Python library introduces missing values in different patterns to time series

A quick example:

```
>>> from pygrinder import mcar
>>> mcar(data_dict['set-a'].values,0.1)
array([[ nan,  nan,  nan, ...,  nan, 71.8,  nan],
       [ nan,  nan,  nan, ...,  nan, 71.8,  nan],
       [ nan,  nan,  nan, ..., 15.3, 71.8,  nan],
       ...,
       [ nan,  nan,  nan, ...,  nan,  nan,  nan],
       [ nan,  nan,  nan, ...,  nan,  nan,  nan],
       [ nan,  nan,  nan, ...,  nan,  nan,  nan]])
```

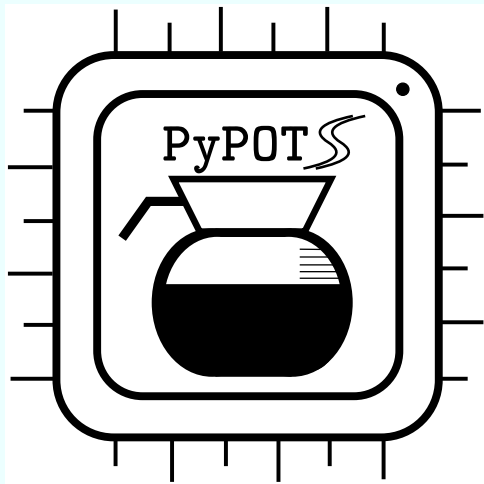



BenchPOTS

A Python library includes standard preprocessing pipelines for 172 public time series datasets

A quick example:

```
>>> import benchpots
>>> benchpots.datasets.preprocess_physionet2012(subset="all", rate=0.1)
2024-08-01 15:05:17 [INFO]: Total sample number: 11988
2024-08-01 15:05:17 [INFO]: Training set size: 7671 (63.99%)
2024-08-01 15:05:17 [INFO]: Validation set size: 1918 (16.00%)
2024-08-01 15:05:17 [INFO]: Test set size: 2399 (20.01%)
2024-08-01 15:05:17 [INFO]: Number of steps: 48
2024-08-01 15:05:17 [INFO]: Number of features: 37
.....
```



PyPOTS [1]

A Python toolkit with 30+ time series neural networks integrated.

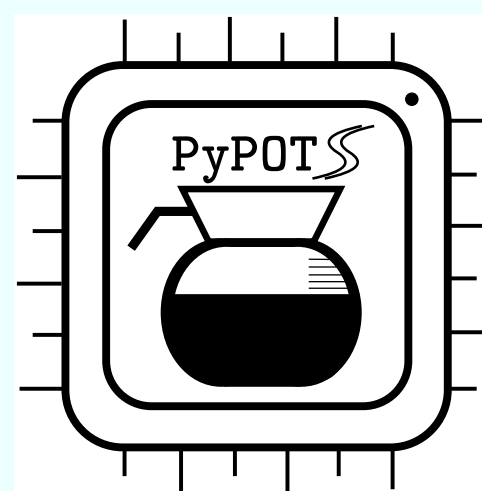
A quick example:

```
>>> from pypots.imputation import SAITS
>>> saits = SAITS(n_steps=48, n_features=37, n_layers=2,
d_model=256, n_heads=4, d_k=64, d_v=64, d_ffn=128,
dropout=0.1, epochs=10)
>>> saits.fit(train_set, val_set)
>>> saits.predict(test_set)
```

[1] Wenjie Du. 2023. PyPOTS: A Python toolbox for data mining on partially-observed time series. arXiv preprint arXiv:2305.18811. <https://arxiv.org/abs/2305.18811>

The task types are abbreviated as follows: **IMPU** : Imputation; **FORE** : Forecasting; **CLAS** : Classification; **CLUS** : Clustering; **ANOD** : Anomaly Detection. The paper references and links are all listed at the bottom of this file.

Type	Algo	IMPU	FORE	CLAS	CLUS	ANOD	Year - Venue
Neural Net	ImputeFormer👤 ^[2]	✓					2024 - KDD
Neural Net	iTransformer👤 ^[3]	✓					2024 - ICLR
Neural Net	SAITS ^[1]	✓					2023 - ESWA
Neural Net	FreTS👤 ^[4]	✓					2023 - NeurIPS
Neural Net	Koopa👤 ^[5]	✓					2023 - NeurIPS
Neural Net	Crossformer👤 ^[6]	✓					2023 - ICLR
Neural Net	TimesNet ^[7]	✓					2023 - ICLR
Neural Net	PatchTST👤 ^[8]	✓					2023 - ICLR
Neural Net	ETSformer👤 ^[9]	✓					2023 - ICLR
Neural Net	MICN👤 ^[10]	✓					2023 - ICLR
Neural Net	DLinear👤 ^[11]	✓					2023 - AAAI
Neural Net	TIDE👤 ^[12]	✓					2023 - TMLR
Neural Net	SCINet👤 ^[13]	✓					2022 - NeurIPS
Neural Net	Nonstationary Tr.👤 ^[14]	✓					2022 - NeurIPS
Neural Net	FILM👤 ^[15]	✓					2022 - NeurIPS
Neural Net	RevIN_SCINet👤 ^[16]	✓					2022 - ICLR
Neural Net	Pyraformer👤 ^[17]	✓					2022 - ICLR
Neural Net	Raindrop ^[18]			✓			2022 - ICLR
Neural Net	FEDformer👤 ^[19]	✓					2022 - ICML
Neural Net	Autoformer👤 ^[20]	✓					2021 - NeurIPS
Neural Net	CSDI ^[21]	✓	✓				2021 - NeurIPS
Neural Net	Informer👤 ^[22]	✓					2021 - AAAI
Neural Net	US-GAN ^[23]	✓					2021 - AAAI
Neural Net	CRL ^[24]				✓		2021 - AAAI
Probabilistic	BTTF ^[25]		✓				2021 - TPAMI
Neural Net	StemGNN👤 ^[26]	✓					2020 - NeurIPS
Neural Net	Reformer👤 ^[27]	✓					2020 - ICLR
Neural Net	GP-VAE ^[28]	✓					2020 - AISTATS
Neural Net	VaDER ^[29]				✓		2019 - GigaSci.
Neural Net	M-RNN ^[30]	✓					2019 - TBME
Neural Net	BRITS ^[31]	✓		✓			2018 - NeurIPS
Neural Net	GRU-D ^[32]	✓		✓			2018 - Sci. Rep.
Neural Net	TCN👤 ^[33]	✓					2018 - arXiv
Neural Net	Transformer👤 ^[34]	✓					2017 - NeurIPS
Naive	Lerp	✓					
Naive	LOCF/NOCB	✓					
Naive	Mean	✓					
Naive	Median	✓					



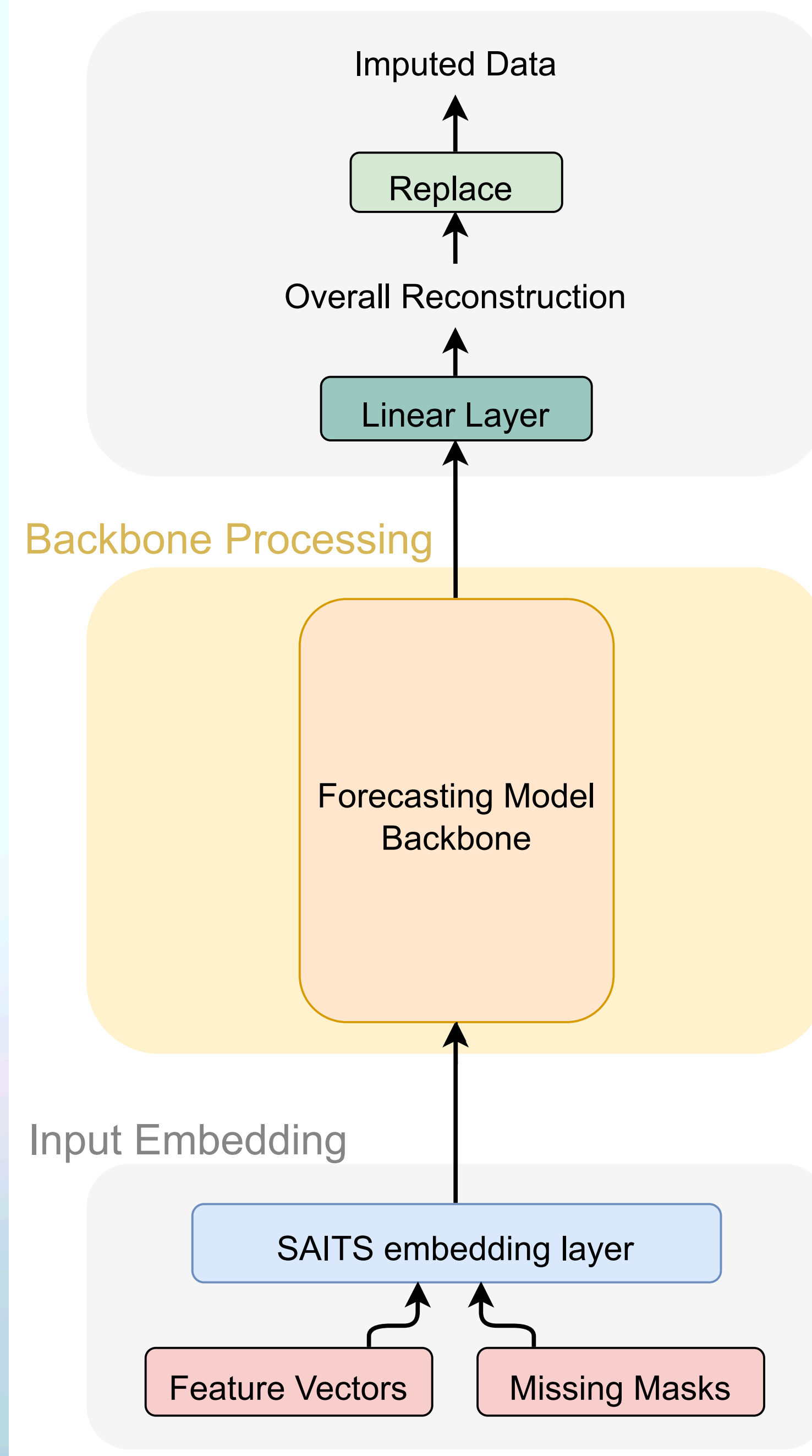
The adaptation method

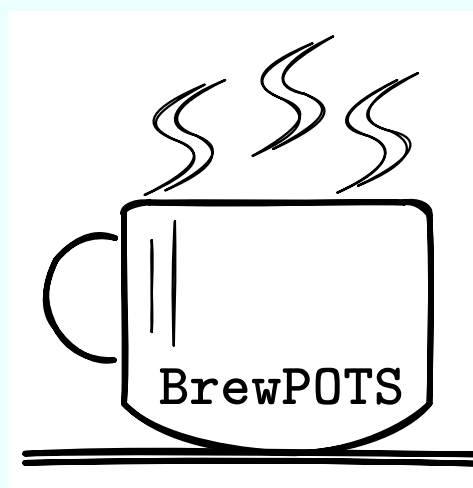
General time series deep learning models don't take POTS into consideration.

To enable them applicable to POTS data, PyPOTS framework adopts the embedding and training method of SAITS^[1] to refactor them

[1] Wenjie Du, David Côté, and Yan Liu. 2023. "SAITS: Self-attention-based imputation for time series." Expert Systems with Applications 219 (2023): 119619. <https://arxiv.org/abs/2202.08516>

Output Projection





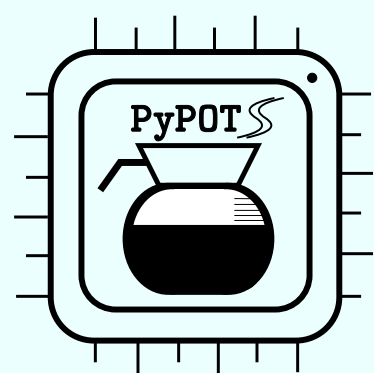
BrewPOTS

Tutorials are necessary for using these tools of PyPOTS ecosystem. PyPOTS tutorials are released in a single repo, and you can find them in BrewPOTS.

[1] <https://github.com/WenjieDu/BrewPOTS>

[2] https://colab.research.google.com/drive/1HEFjylEy05-r47jRy0H9jiS_WhD0UWmQ#scrollTo=xe-rPZzC9CoW

Table of contents	
👋 Quick-start Tutorials for PyPOTS are Here!	⋮
Dependency Installation	⋮
💿 Preparing the PhysioNet-2012 dataset for this tutorial	⋮
🌟 Imputation Models	⋮
🚀 An example of SAITS for imputation	⋮
🚀 An example of Transformer for imputation	⋮
🚀 An example of TimesNet for imputation	⋮
🚀 An example of CSDI for imputation	⋮
🚀 An example of US-GAN for imputation	⋮
🚀 An example of GP-VAE for imputation	⋮
🚀 An example of BRITS for imputation	⋮
🚀 An example of M-RNN for imputation	⋮
🚀 An example of LOCF for imputation	⋮
🌟 Clustering Models	⋮
🚀 An example of CRLI for clustering	⋮
🚀 An example of VaDER for clustering	⋮
🌟 Forecasting Models	⋮
🚀 An example of BTTF for forecasting	⋮
🌟 Classification Models	⋮
🚀 An example of BRITS for classification	⋮
🚀 An example of GRUD for classification	⋮
🚀 An example of Raindrop for classification	⋮

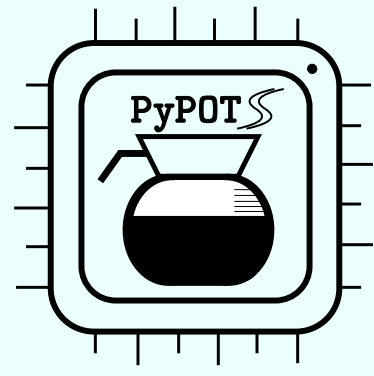


Our two recent work about TSI

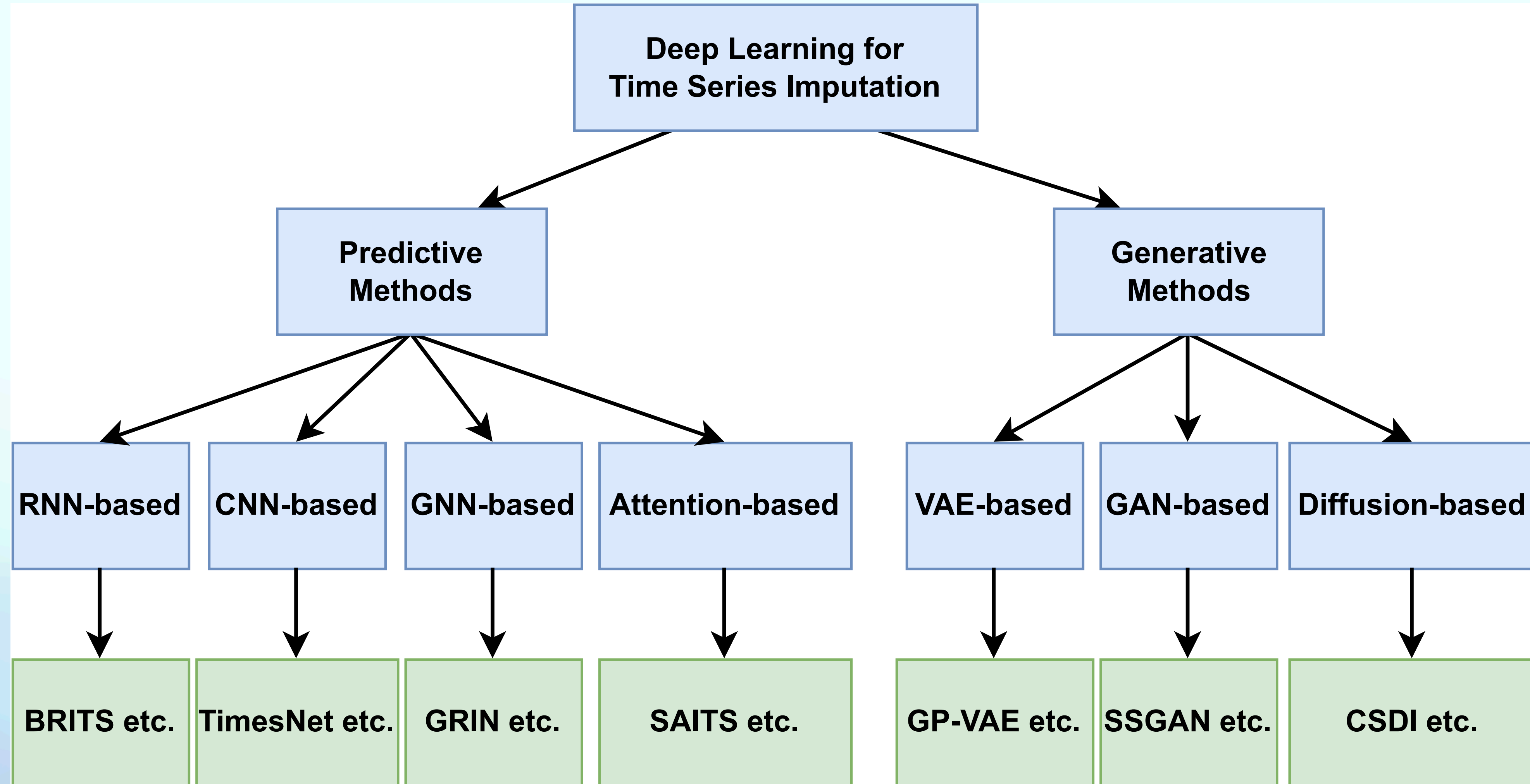
1. Survey on Time Series Imputation ^[1];
2. Benchmark for Time Series Imputation ^[2];

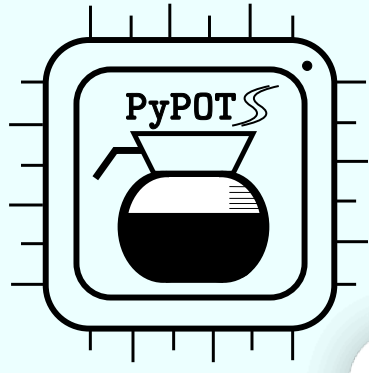
[1] Jun Wang, Wenjie Du, Wei Cao, Keli Zhang, Wenjia Wang, Yuxuan Liang, and Qingsong Wen. 2024. Deep learning for multivariate time series imputation: A survey. arXiv preprint arXiv:2402.04059. <https://arxiv.org/abs/2406.12747>

[2] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>



A literature review of TS Imputation ^[1]

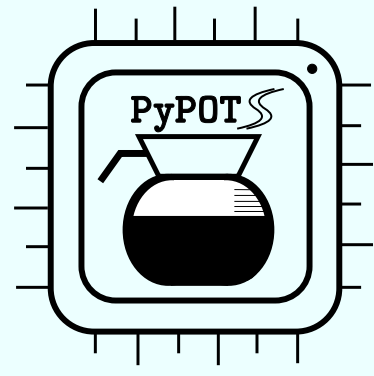




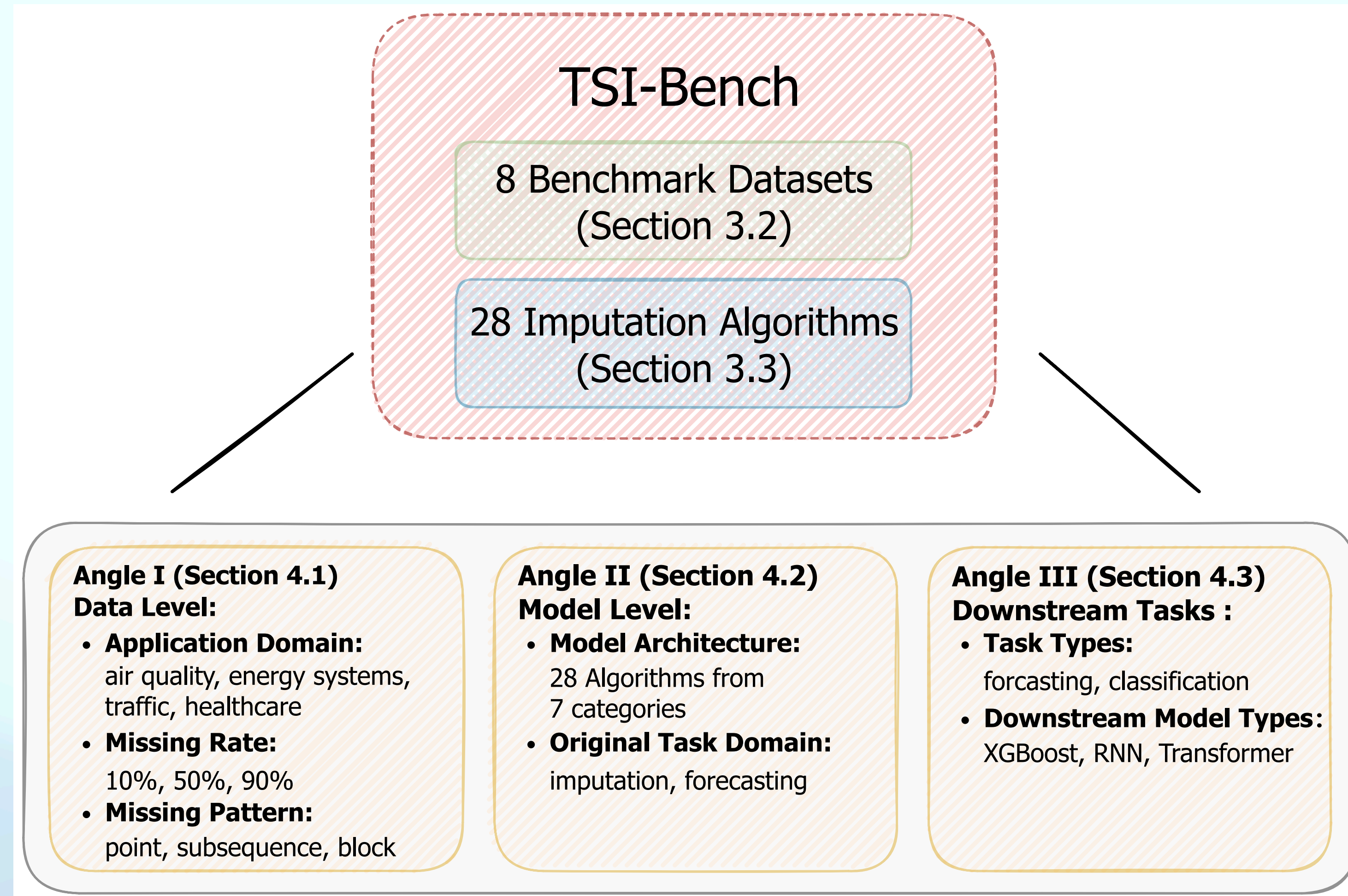
A literature review of TS Imputation ^[1]

Method	Venue	Category	Imputation Uncertainty	Neural Network Architecture	Missing Mechanism
GRU-D [Che <i>et al.</i> , 2018]	Scientific Reports	predictive	✗	RNN	MCAR
M-RNN [Yoon <i>et al.</i> , 2019]	TBME	predictive	✗	RNN	MCAR
BRITS [Cao <i>et al.</i> , 2018]	NeurIPS	predictive	✗	RNN	MCAR
TimesNet [Wu <i>et al.</i> , 2023a]	ICLR	predictive	✗	CNN	MCAR
GRIN [Cini <i>et al.</i> , 2022]	ICLR	predictive	✗	GNN	MCAR / MAR
SPIN [Marisca <i>et al.</i> , 2022]	NeurIPS	predictive	✗	GNN, Attention	MCAR / MAR
CDSA [Ma <i>et al.</i> , 2019]	arXiv	predictive	✗	Attention	MCAR
Transformer [Vaswani <i>et al.</i> , 2017]	NeurIPS	predictive	✗	Attention	MCAR
SAITS [Du <i>et al.</i> , 2023]	ESWA	predictive	✗	Attention	MCAR
DeepMVI [Bansal <i>et al.</i> , 2021]	VLDB	predictive	✗	Attention, CNN	MCAR
NRTSI [Shan <i>et al.</i> , 2023]	ICASSP	predictive	✗	Attention	MCAR
GP-VAE [Fortuin <i>et al.</i> , 2020]	AISTATS	generative	⊙	VAE, CNN	MCAR / MAR
V-RIN [Mulyadi <i>et al.</i> , 2021]	Trans. Cybern.	generative	✓	VAE, RNN	MCAR / MAR
supnotMIWAE [Kim <i>et al.</i> , 2023]	ICML	generative	⊙	VAE	MNAR
GRUI-GAN [Luo <i>et al.</i> , 2018]	NeurIPS	generative	⊙	GAN, RNN	MCAR
E ² GAN [Luo <i>et al.</i> , 2019]	IJCAI	generative	⊙	GAN, RNN	MCAR
NAOMI [Liu <i>et al.</i> , 2019]	NeurIPS	generative	⊙	GAN, RNN	MCAR
SSGAN [Miao <i>et al.</i> , 2021]	AAAI	generative	⊙	GAN, RNN	MCAR
CSDI [Tashiro <i>et al.</i> , 2021]	NeurIPS	generative	⊙	Diffusion, Attention, CNN	MCAR
SSSD [Alcaraz and Strodthoff, 2023]	TMLR	generative	⊙	Diffusion, Attention	MCAR
CSBI [Chen <i>et al.</i> , 2023]	ICML	generative	⊙	Diffusion, Attention	MCAR
MIDM [Wang <i>et al.</i> , 2023]	KDD	generative	⊙	Diffusion, Attention	MCAR
PriSTI [Liu <i>et al.</i> , 2023]	ICDE	generative	⊙	Diffusion, Attention, GNN, CNN	MCAR
DA-TASWDM [Xu <i>et al.</i> , 2023]	CIKM	generative	✓	Diffusion, Attention	MCAR
SPD [Biloš <i>et al.</i> , 2023]	ICML	generative	⊙	Diffusion, Attention	MCAR

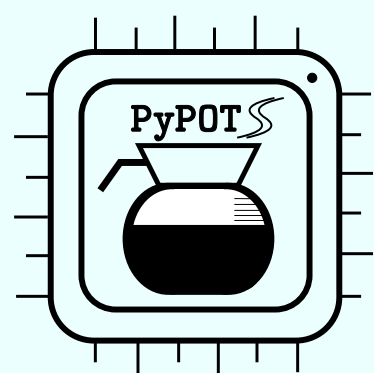
Table 1: Summary of deep learning methods for multivariate time series imputation. ✓ and ⊙ indicate methods capable of accounting for imputation uncertainty, whereas ✗ denotes methods that do not. Furthermore, ✓ denotes that the methods also define the fidelity score to explicitly measure the imputation uncertainty.



TSI-Bench: Benchmarking TS Imputation [1]

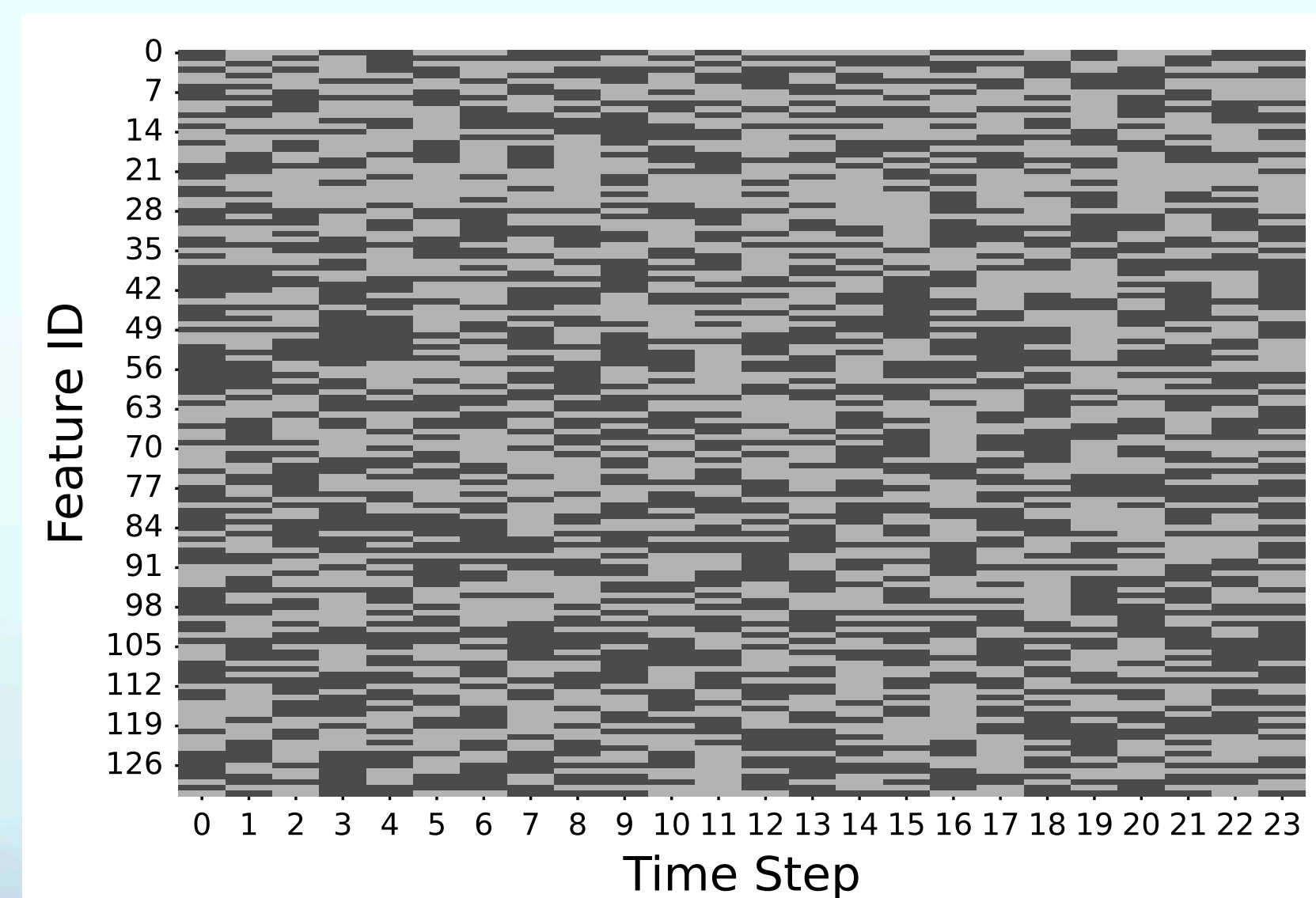


[1] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>

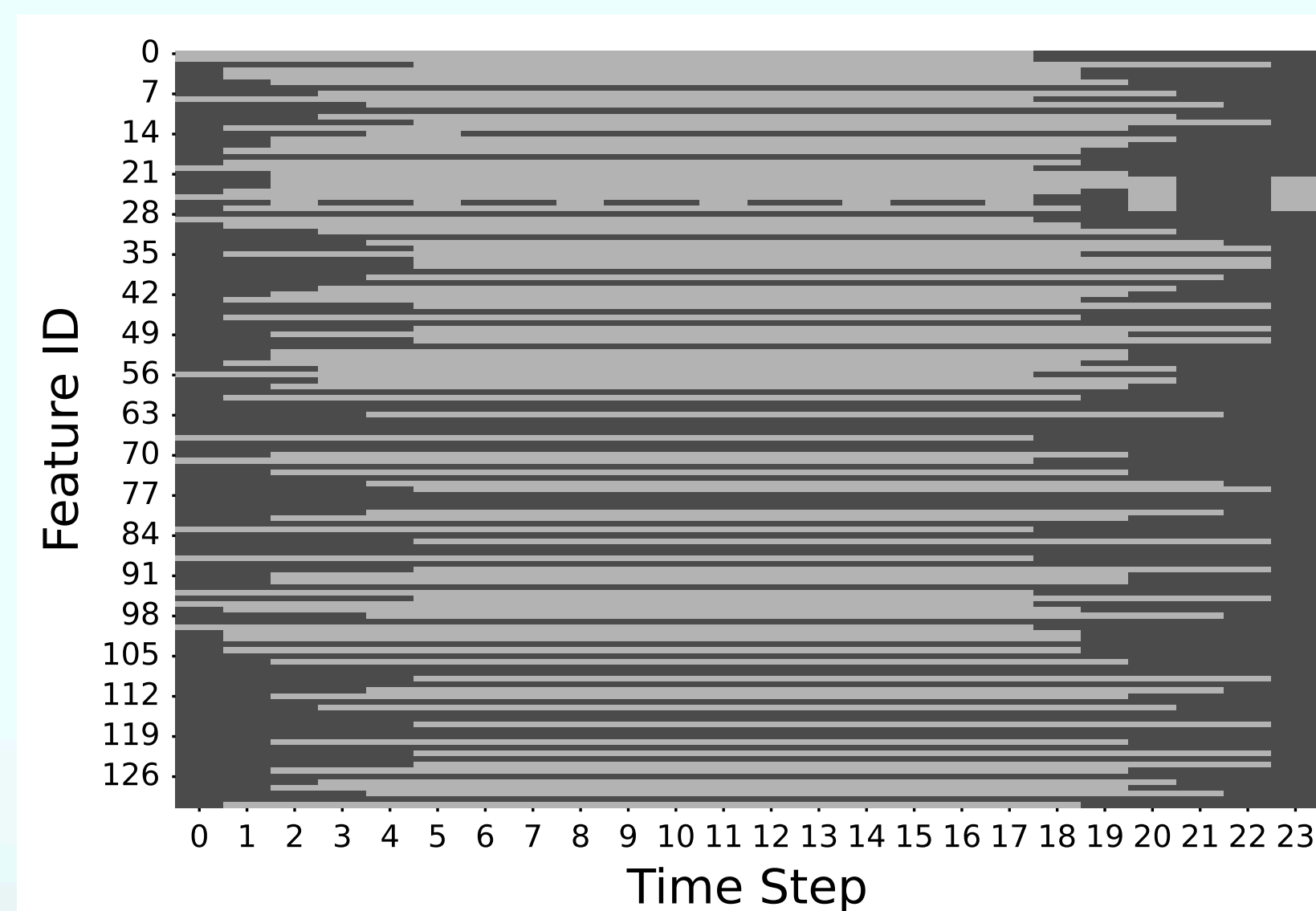


Different missing patterns

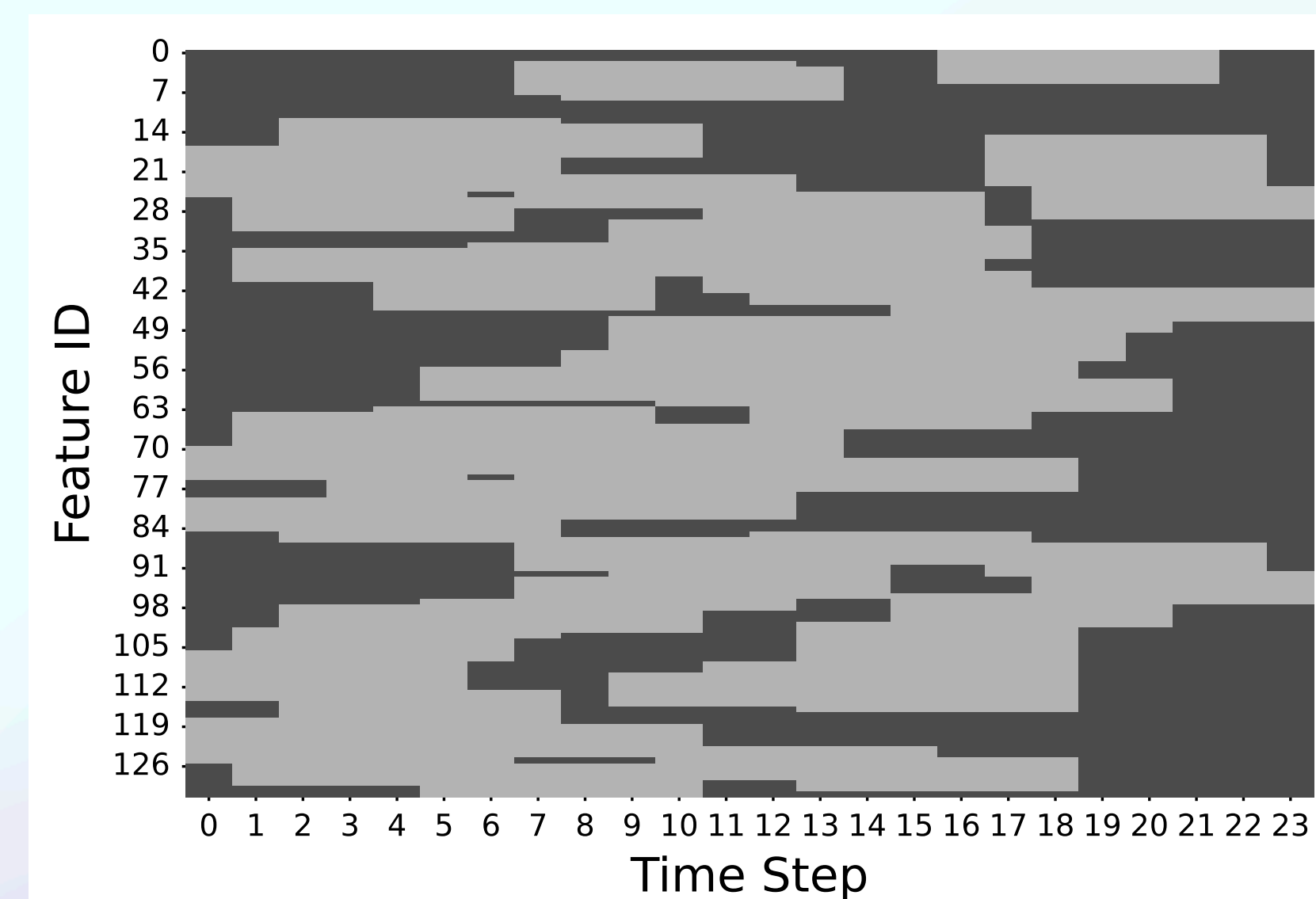
We pick a more visualization friendly perspective, point, sequence, and block



Point Missing

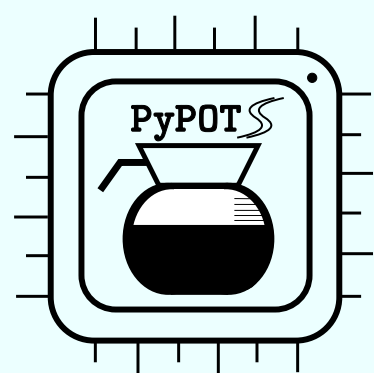


Subsequence Missing



Block Missing

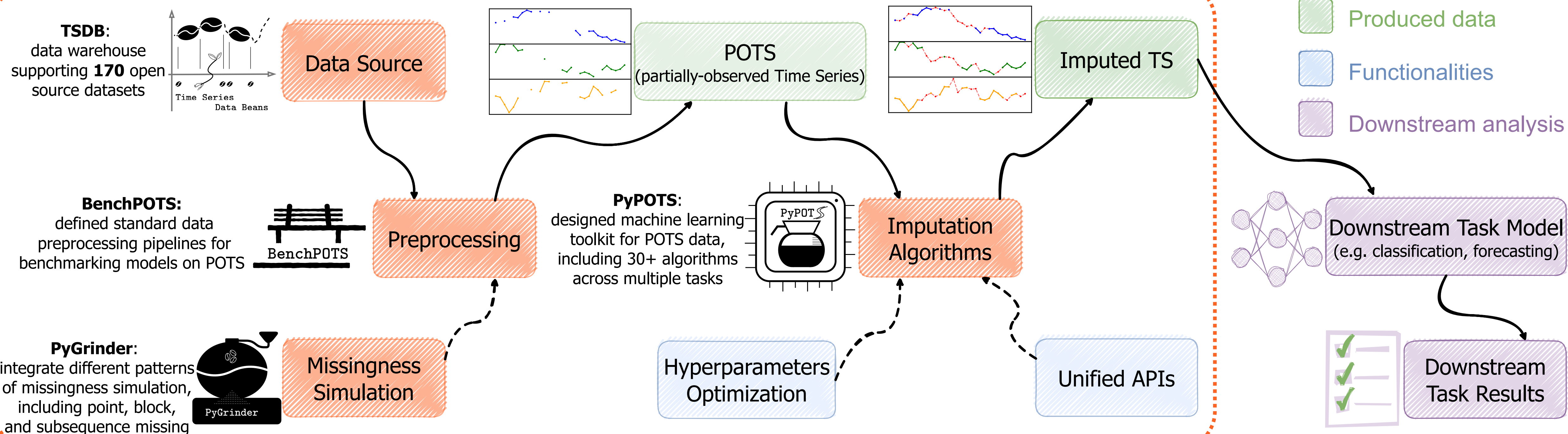
[1] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>



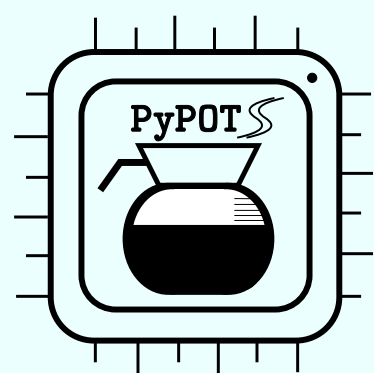
TSI-Bench suite [1]

TSI-Bench Suite

PyPOTS Ecosystem



[1] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>

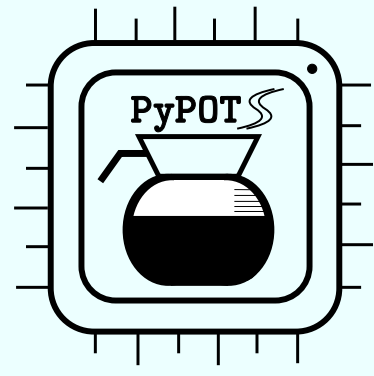


TSI-Bench: Benchmarking TS Imputation [1]

Table 4: Summary of the 28 benchmarked algorithms, separated by **Architecture**

Model	Category	Architecture	Venue	Year
iTransformer [60]	Forecasting	Transformer	ICLR	2024
SAITS [20]	Imputation	Transformer	ESWA	2023
Nonstationary [59]	Forecasting	Transformer	NeurIPS	2022
ETSformer [58]	Forecasting	Transformer	ArXiv	2022
PatchTST [33]	Forecasting	Transformer	ICLR	2023
Crossformer [43]	Forecasting	Transformer	ICLR	2022
Informer [42]	Forecasting	Transformer	AAAI	2021
Autoformer [31]	Forecasting	Transformer	NeurIPS	2021
Pyraformer [57]	Forecasting	Transformer	ICLR	2021
Transformer [56]	Forecasting	Transformer	NeurIPS	2017
BRITS [19]	Imputation	RNN	NeurIPS	2018
MRNN [61]	Imputation	RNN	TBME	2018
GRUD [62]	Imputation	RNN	Scientific reports	2018
TimesNet [13]	Generic	CNN	ICLR	2023
MICN [63]	Forecasting	CNN	ICLR	2022
SCINet [64]	Forecasting	CNN	NeurIPS	2022
StemGNN [34]	Forecasting	GNN	NeurIPS	2020
FreTS [67]	Forecasting	MLP	NeurIPS	2024
Koopa [66]	Forecasting	MLP	NeurIPS	2024
DLinear [44]	Forecasting	MLP	AAAI	2023
FiLM [65]	Forecasting	MLP	NeurIPS	2022
CSDI [30]	Imputation	Diffusion	NeurIPS	2021
US-GAN [68]	Imputation	GAN	AAAI	2021
GP-VAE [28]	Imputation	VAE	AISTATS	2020
Mean	Imputation	Traditional	-	-
Median	Imputation	Traditional	-	-
LOCF	Imputation	Traditional	-	-
Linear	Imputation	Traditional	-	-

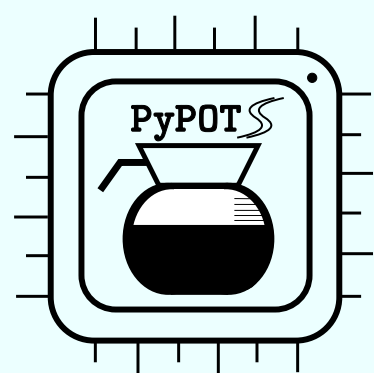
[1] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>



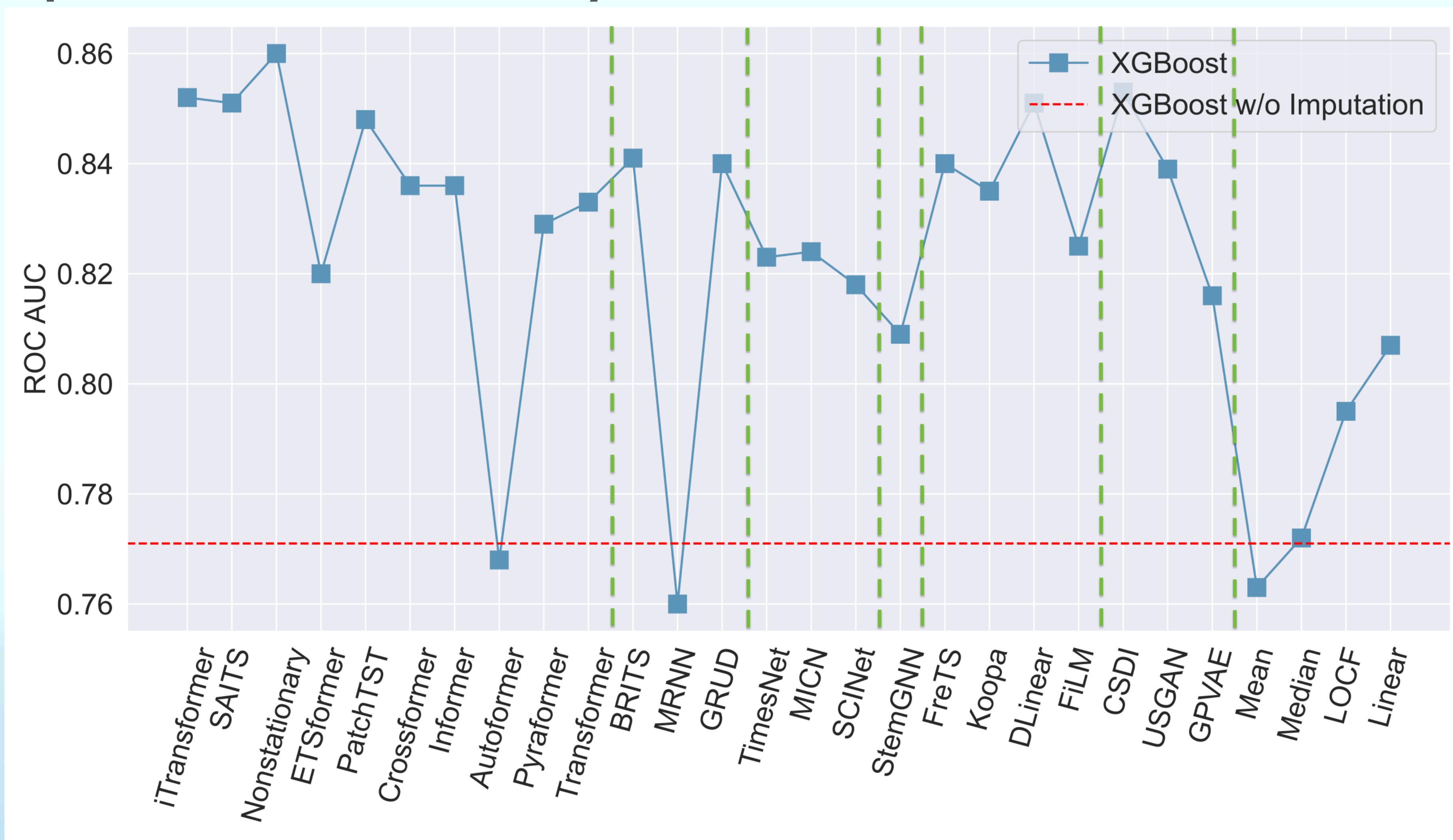
TSI-Bench: Benchmarking TS Imputation [1]

	BeijingAir	ItalyAir	PeMS	Pedestrian	ETT_h1	Electricity	PhysioNet2012	PhysioNet2019
iTransformer	0.123 (0.005)	0.223 (0.014)	0.226 (0.001)	0.148 (0.005)	0.263 (0.004)	0.571 (0.178)	0.379 (0.002)	0.462 (0.006)
SAITS	0.155 (0.004)	0.185 (0.010)	0.287 (0.001)	0.131 (0.006)	0.144 (0.006)	1.377 (0.026)	0.257 (0.019)	0.352 (0.005)
Nonstationary	0.209 (0.002)	0.266 (0.007)	0.331 (0.017)	0.453 (0.024)	0.359 (0.013)	0.213 (0.014)	0.410 (0.002)	0.458 (0.001)
ETSformer	0.187 (0.002)	0.259 (0.004)	0.347 (0.006)	0.207 (0.011)	0.227 (0.007)	0.412 (0.005)	0.373 (0.003)	0.451 (0.005)
PatchTST	0.198 (0.011)	0.274 (0.026)	0.330 (0.013)	0.126 (0.003)	0.240 (0.013)	0.550 (0.039)	0.301 (0.011)	0.420 (0.007)
Crossformer	0.184 (0.004)	0.246 (0.011)	0.337 (0.007)	0.119 (0.005)	0.232 (0.008)	0.540 (0.034)	0.525 (0.202)	0.378 (0.007)
Informer	0.148 (0.002)	0.205 (0.008)	0.302 (0.003)	0.154 (0.010)	0.167 (0.006)	1.291 (0.031)	0.297 (0.003)	0.403 (0.002)
Autoformer	0.257 (0.012)	0.295 (0.008)	0.598 (0.074)	0.197 (0.008)	0.267 (0.008)	0.748 (0.027)	0.417 (0.009)	0.476 (0.002)
Pyraformer	0.178 (0.004)	0.217 (0.006)	0.285 (0.003)	0.153 (0.012)	0.182 (0.008)	1.096 (0.033)	0.294 (0.002)	0.387 (0.004)
Transformer	0.142 (0.001)	0.191 (0.010)	0.294 (0.002)	0.136 (0.009)	0.178 (0.015)	1.316 (0.036)	0.259 (0.006)	0.341 (0.002)
BRITS	0.127 (0.001)	0.235 (0.007)	0.271 (0.000)	0.149 (0.005)	0.145 (0.002)	0.971 (0.016)	0.297 (0.001)	0.355 (0.001)
MRNN	0.568 (0.002)	0.638 (0.003)	0.624 (0.000)	0.735 (0.001)	0.789 (0.019)	1.824 (0.005)	0.708 (0.029)	0.778 (0.015)
GRUD	0.233 (0.002)	0.368 (0.012)	0.355 (0.002)	0.204 (0.008)	0.325 (0.004)	0.976 (0.015)	0.450 (0.004)	0.471 (0.001)
TimesNet	0.230 (0.010)	0.280 (0.004)	0.312 (0.001)	0.157 (0.008)	0.254 (0.008)	1.011 (0.016)	0.353 (0.003)	0.394 (0.003)
MICN	0.203 (0.001)	0.283 (0.004)	0.281 (0.003)	/	0.267 (0.010)	0.392 (0.006)	0.378 (0.013)	0.461 (0.007)
SCINet	0.191 (0.011)	0.288 (0.010)	0.487 (0.101)	0.149 (0.012)	0.246 (0.019)	0.581 (0.015)	0.341 (0.005)	0.427 (0.002)
StemGNN	0.161 (0.002)	0.260 (0.008)	0.493 (0.079)	0.127 (0.006)	0.248 (0.012)	1.360 (0.078)	0.331 (0.001)	0.416 (0.002)
FreTS	0.211 (0.008)	0.273 (0.008)	0.396 (0.027)	0.138 (0.004)	0.262 (0.029)	0.718 (0.043)	0.315 (0.008)	0.406 (0.017)
Koopa	0.363 (0.108)	0.307 (0.041)	0.532 (0.122)	0.173 (0.020)	0.435 (0.132)	1.309 (0.531)	0.413 (0.007)	0.451 (0.019)
DLinear	0.215 (0.016)	0.242 (0.009)	0.362 (0.009)	0.179 (0.004)	0.227 (0.006)	0.519 (0.008)	0.370 (0.000)	0.432 (0.001)
FiLM	0.318 (0.010)	0.340 (0.011)	0.784 (0.064)	0.413 (0.010)	0.583 (0.008)	0.834 (0.031)	0.458 (0.001)	0.494 (0.003)
CSDI	0.102 (0.010)	0.539 (0.418)	0.238 (0.047)	0.231 (0.064)	0.151 (0.008)	1.483 (0.459)	0.252 (0.002)	0.408 (0.019)
US-GAN	0.137 (0.002)	0.264 (0.012)	0.296 (0.001)	0.151 (0.016)	0.458 (0.590)	0.938 (0.009)	0.310 (0.003)	0.358 (0.002)
GP-VAE	0.240 (0.006)	0.369 (0.012)	0.341 (0.007)	0.319 (0.010)	0.329 (0.017)	1.152 (0.074)	0.445 (0.006)	0.562 (0.004)
Mean	0.721	0.574	0.798	0.728	0.737	0.422	0.708	0.762
Median	0.681	0.518	0.778	0.667	0.71	0.408	0.69	0.747
LOCF	0.188	0.233	0.375	0.257	0.315	0.104	0.449	0.478
Linear	0.112	0.135	0.211	0.167	0.197	0.065	0.366	0.387

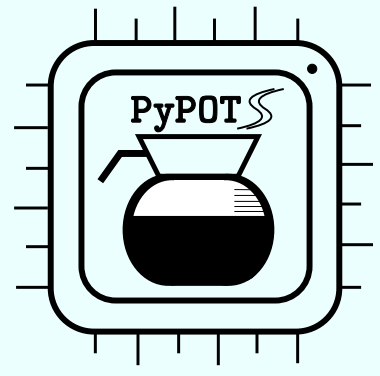
[1] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>



Imputation really matters! [1]

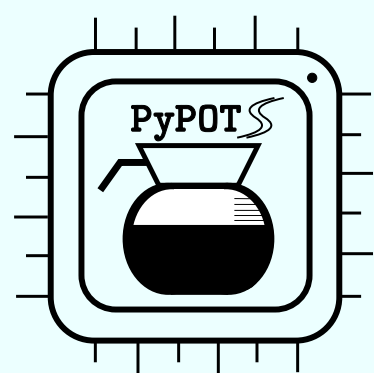


[1] Wenjie Du, Jun Wang, Linglong Qian, Yiyuan Yang, Fanxing Liu, Zepu Wang, Zina Ibrahim, Haoxin Liu, Zhiyuan Zhao, Yingjie Zhou, Wenjia Wang, Kaize Ding, Yuxuan Liang, B. Aditya Prakash, Qingsong Wen. 2024. "TSI-Bench: Benchmarking Time Series Imputation." arXiv preprint arXiv:2406.12747. <https://arxiv.org/abs/2406.12747>



Imputation, a way to train and measure

Imputation is not only a time series analysis task, more importantly, it is a way for model training and model evaluation.



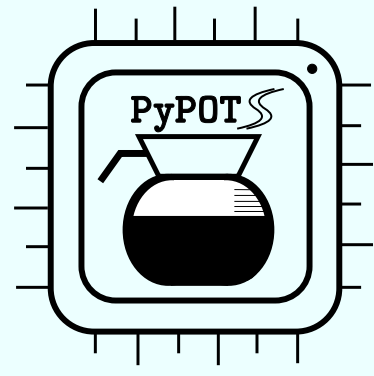
Masking tasks in recent LLM4TS work

MOMENT [1]

Masked Representation Learning. Masked pre-training is a widely-used self-supervised learning task where a model learns to accurately reconstruct masked portions of its input. Masked language (Devlin et al., 2019; Raffel et al., 2020) and image modeling (Xie et al., 2022; Li et al., 2023b) have been successfully utilized to learn models from vast quantities of unlabeled data, which can generalize to a variety of downstream tasks.

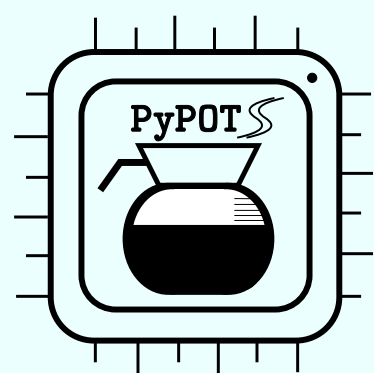
For time series data, prior work has primarily focused on contrastive representation learning (Yue et al., 2022; Eldele et al., 2021; Franceschi et al., 2019). The goal of contrastive learning is to learn a representation space where “positive” pairs of time series are close while “negative” pairs are far apart. However, the notion of positive and negative pairs is subjective and data-dependent, and popular transformations such as flipping and cropping invariance may not be appropriate for time series data (Yue et al., 2022). In contrast, some studies mask portions of time series using zeros and learn a model to reconstruct them (Nie et al., 2023; Zerveas et al., 2021; Dong et al., 2023; Li et al., 2023c).

Representation learning via masking is well-suited to all the downstream tasks we care about, especially forecasting and imputation, as they are instances of the masked reconstruction problem. Owing to its simplicity and success in vision and language domains, we use the masked prediction task to pre-train our model, using a special embedding (see [MASK] in Fig. 3) to mask time series patches instead of zeros.



Summary

1. Imputation helps with the downstream tasks;
2. Imputation is a way to train and measure/eval a model learning from POTS;
3. PyPOTS Ecosystem including Python libraries dedicated to machine learning for POTS;
4. We need more advanced algorithms to understand POTS data to accomplish multiple learning tasks end2end, e.g LLM for POTS;



Credit for the work



Jun Wang

Ph.D. candidate@HKUST



Linglong Qian

Ph.D. candidate@KCL



Yiyuan Yang

Ph.D. candidate@Oxford



Zina Ibrahim

Associate Professor@KCL



Qingsong Wen

Chief Scientist@Squirrel AI

Thank you!